

Redes Neurais Artificiais



Gabriel P. Oliveira

Inteligência Artificial
2026/1

Redes Neurais



1.

Multilayer Perceptron (MLP)

Quando adicionamos mais camadas ao Perceptron

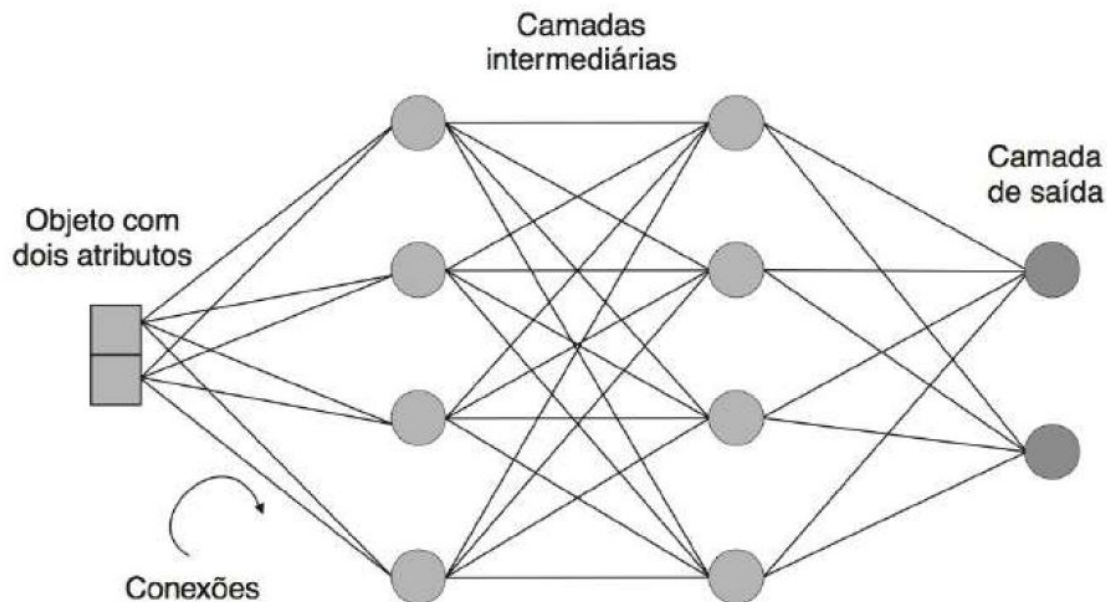
Multilayer Perceptron (MLP)

Para resolver problemas não linearmente separáveis utilizando RNAs, a alternativa mais utilizada é **adicionar uma ou mais camadas intermediárias**.

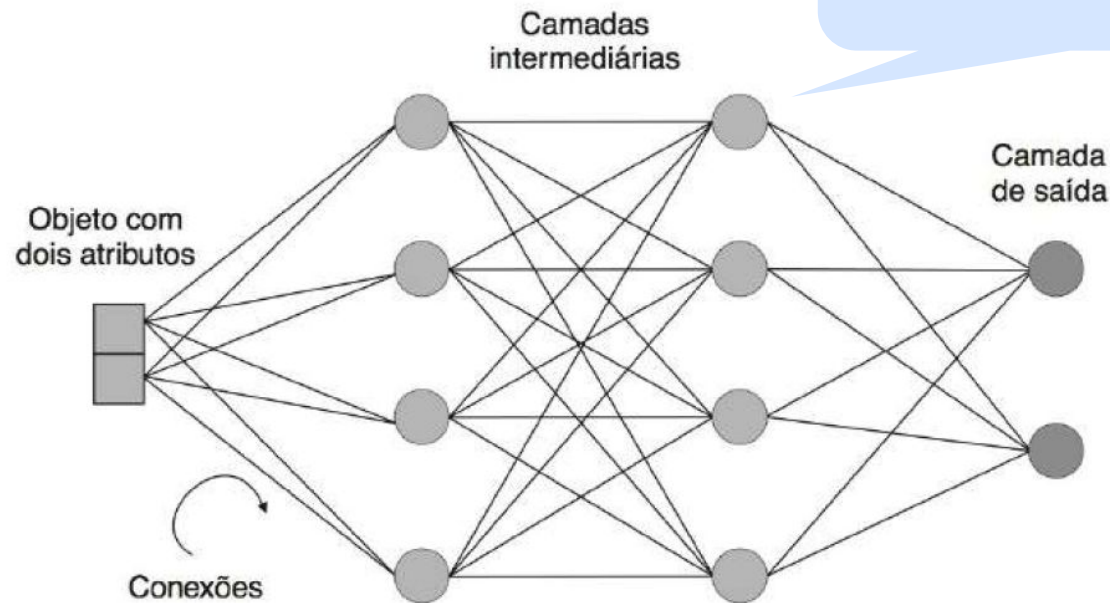
Multilayer Perceptron (MLP)

MLP → uma ou mais camadas intermediárias de neurônios e uma camada de saída

- ▷ Geralmente usam arquitetura **completamente conectada**

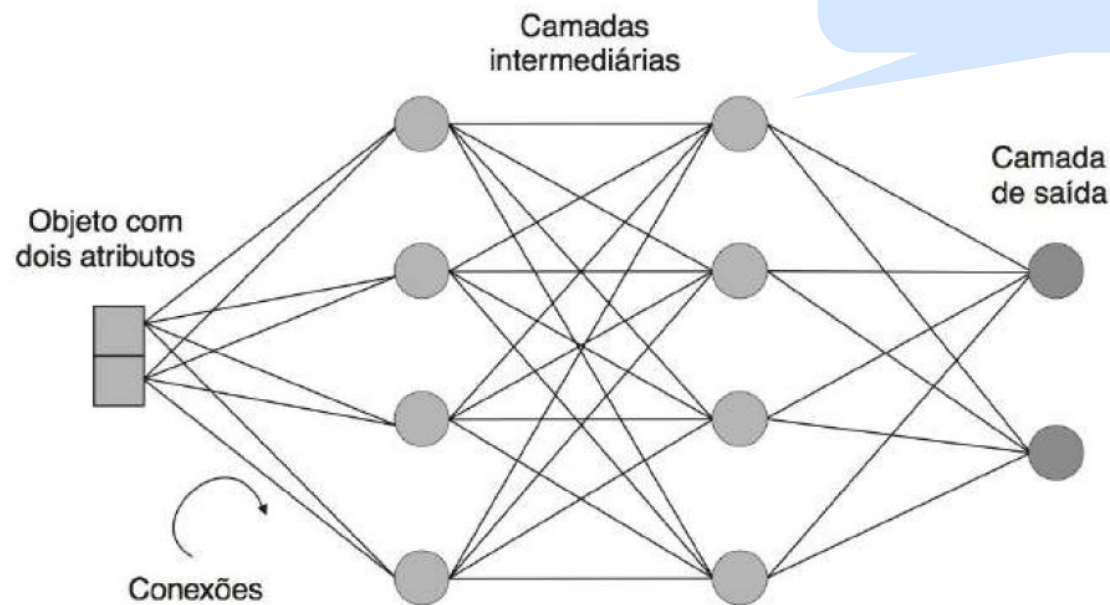


Multilayer Perceptron (MLP)



Redes multicamadas usam funções de ativação **não lineares** (i.e., função sigmoial)

Multilayer Perceptron (MLP)

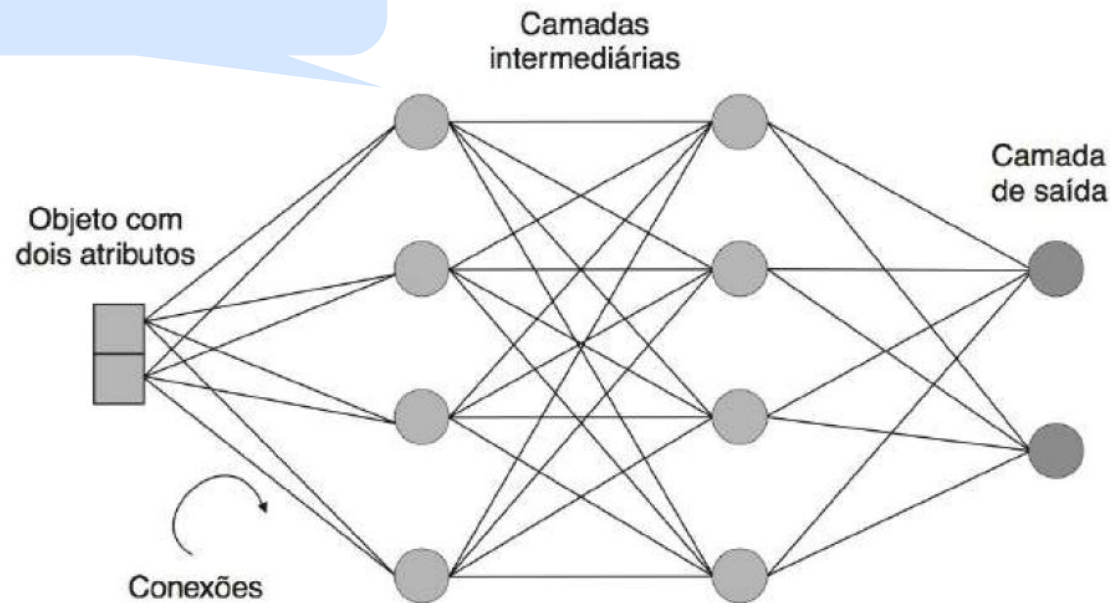


Redes multicamadas usam funções de ativação **não lineares** (i.e., função sigmoideal)

Uma rede multicamadas com funções de ativação lineares é **equivalente a uma rede de uma única camada**

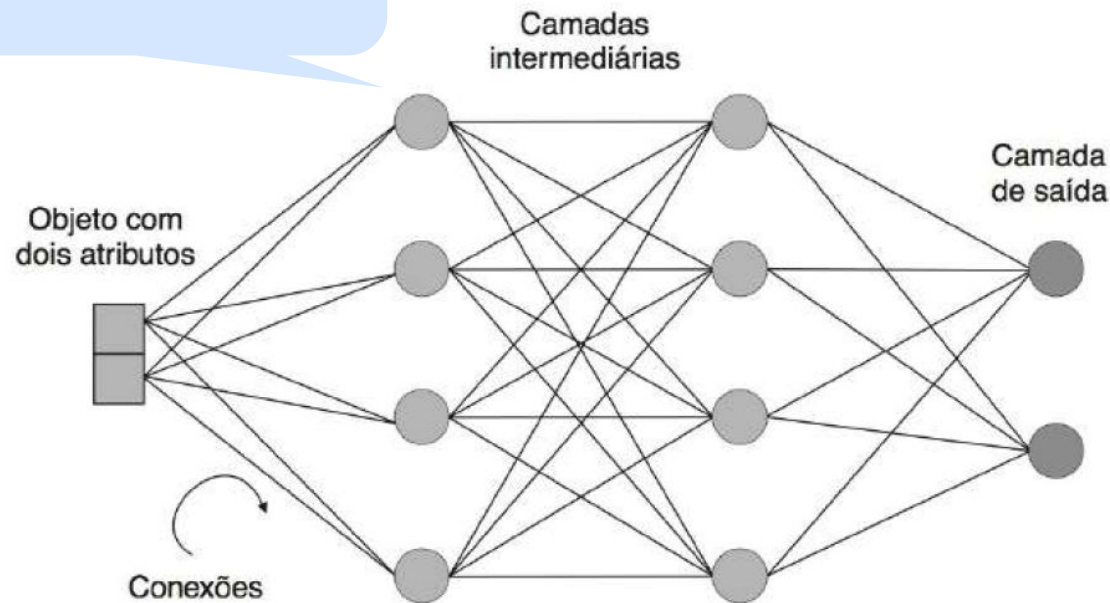
Multilayer Perceptron (MLP)

Em uma MLP, cada neurônio realiza uma função específica



Multilayer Perceptron (MLP)

Em uma MLP, cada neurônio realiza uma função específica



À medida em que o processamento avança de uma camada para outra, o processamento se torna mais complexo

Multilayer Perceptron (MLP)

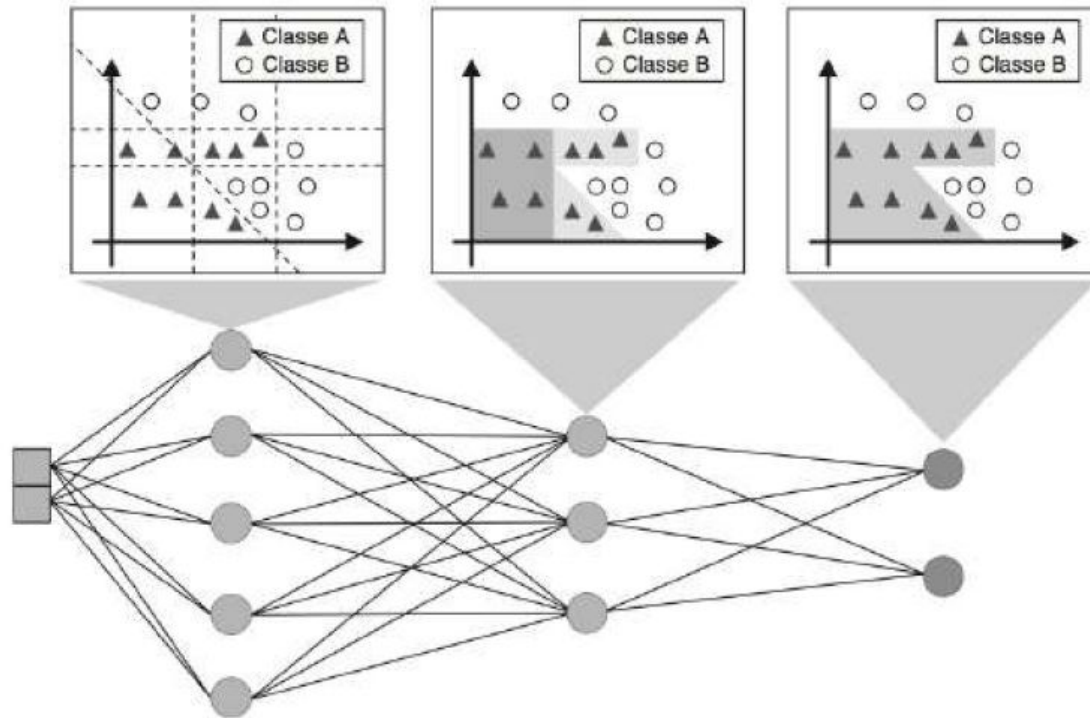
Primeira camada → cada neurônio aprende uma função que define um **hiperplano**

Separa o espaço de entrada em duas partes

Segunda camada → combina um grupo de hiperplanos da camada anterior, formando **regiões convexas**

Camada seguinte → combinam regiões convexas em regiões de **formato arbitrário**

Multilayer Perceptron (MLP)



É a combinação das funções desempenhadas por cada neurônio que define a função associada à RNA

Multilayer Perceptron (MLP)

E como fazemos para treinar redes multicamadas?

→ Algoritmo backpropagation!

2.

Algoritmo Backpropagation

O algoritmo que tornou as redes neurais profundas possíveis

O Algoritmo Backpropagation

- ▷ Proposto por **Rumelhart, Hinton e Williams (1986)**
- ▷ Base do aprendizado supervisionado em redes MLP

nature

Letter | Published: 09 October 1986

Learning representations by back-propagating errors

[David E. Rumelhart](#), [Geoffrey E. Hinton](#) & [Ronald J. Williams](#)

[Nature](#) **323**, 533–536 (1986) | [Cite this article](#)

247k Accesses | **25k** Citations | **818** Altmetric | [Metrics](#)

Disponível em: <https://www.nature.com/articles/323533a0>

Backpropagation: a intuição

É um algoritmo baseado no **aprendizado supervisionado** por correção de erros, e possui duas fases:

1. Propagação (fase *forward*)
2. Retropropagação (fase *backward*)

Backpropagation: a intuição

É um algoritmo baseado no **aprendizado supervisionado** por correção de erros, e possui duas fases:

1. Fase para frente (*forward pass*)
2. Fase para trás (*backward pass*)

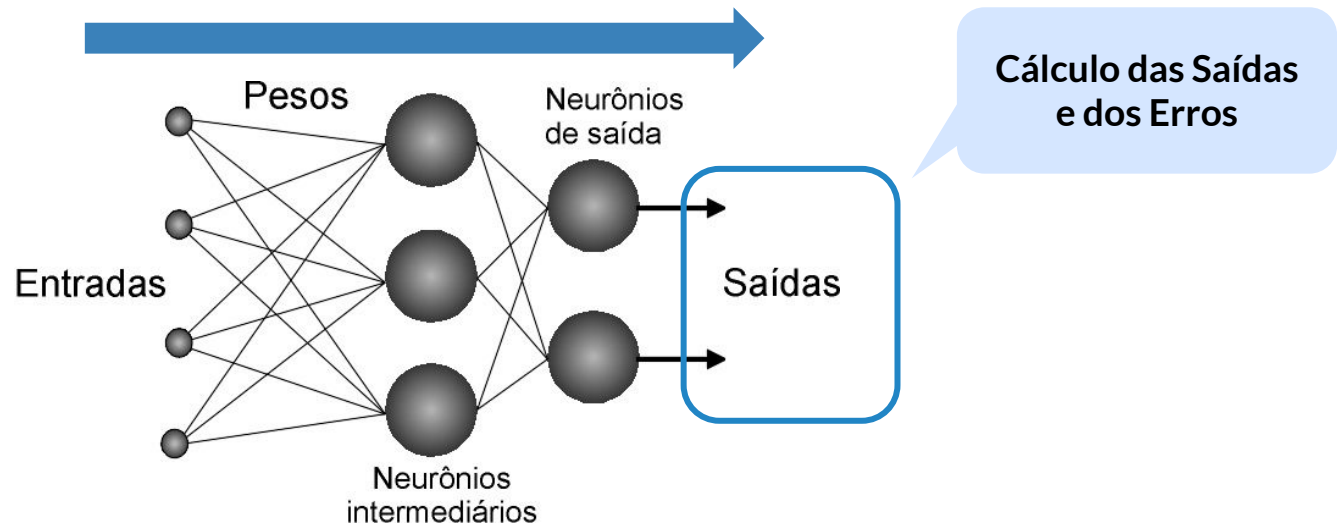
A intuição é simples:

- (1) Imagine que você errou a classificação de uma instância.
- (2) Você volta, entende onde errou e tenta não repetir o erro.

Backpropagation: a intuição

1. Fase para frente (*forward*)

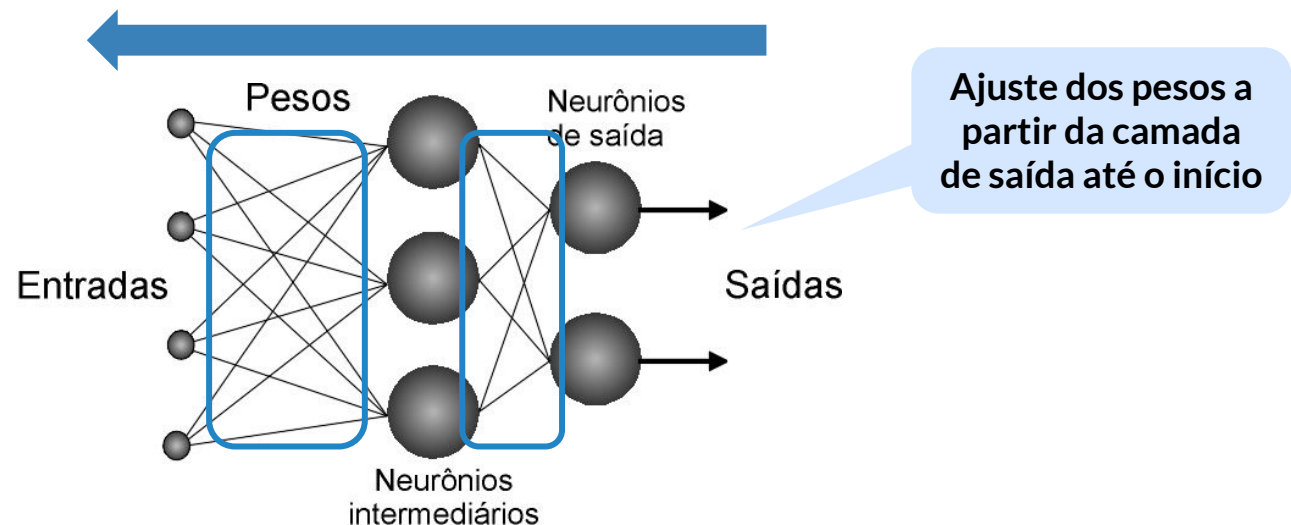
- Os dados entram pela rede
- Cada neurônio calcula sua saída e passa para a próxima camada
- A rede produz uma resposta ($\hat{y}$)



Backpropagation: a intuição

2. Fase para trás (*backward*)

- O erro “volta” pela rede, camada por camada
- Os pesos são ajustados com base no erro

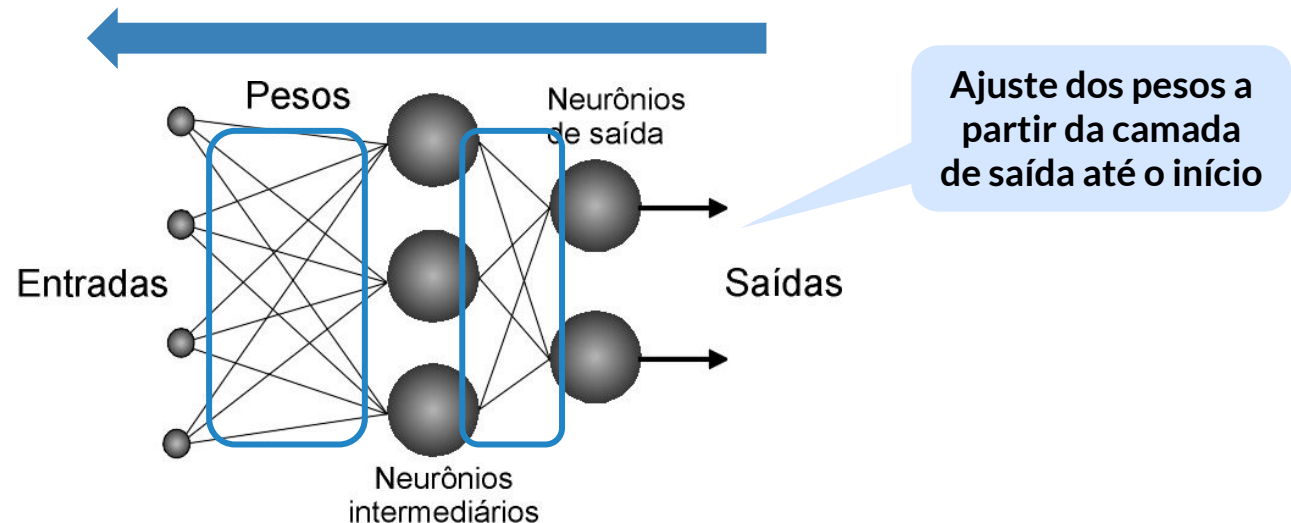


Backpropagation: a intuição

2. Fase para trás (*backward*)

- O erro “volta” pela rede, camada por camada
- Os pesos são ajustados com base no erro

como?



Backpropagation – Treinamento

Algoritmo 7.2 Algoritmo de treinamento *back-propagation*

Entrada: Um conjunto de n objetos de treinamento

Saída: Rede MLP com valores dos pesos ajustados

```
1 Inicializar pesos da rede com valores aleatórios
2 Inicializar  $erro_{total} = 0$ 
3 repita
4   para cada objeto  $x_i$  do conjunto de treinamento faça
5     para cada camada da rede, a partir da primeira camada intermediária faça
6       para cada neurônio  $n_{jl}$  da camada atual faça
7         Calcular valor da saída produzida pelo neurônio,  $\hat{f}$ 
8       fim
9     fim
10    Calcular  $erro_{parcial} = y - \hat{f}$ 
11    para cada camada da rede, a partir da camada de saída faça
12      para cada neurônio  $n_{jl}$  da camada atual faça
13        Ajustar pesos do neurônio utilizando Equação 7.3
14      fim
15    fim
16    Calcular  $erro_{total} = erro_{total} + erro_{parcial}$ 
17  fim
18 até  $erro_{total} < \xi$ ;
```

Backpropagation – Treinamento

Algoritmo 7.2 Algoritmo de treinamento *back-propagation*

Entrada: Um conjunto de n objetos de treinamento

Saída: Rede MLP com valores dos pesos ajustados

```
1 Inicializar pesos da rede com valores aleatórios
2 Inicializar  $erro_{total} = 0$ 
3 repita
4   para cada objeto  $x_i$  do conjunto de treinamento faça
5     para cada camada da rede, a partir da primeira camada intermediária faça
6       para cada neurônio  $n_{jl}$  da camada atual faça
7         Calcular valor da saída produzida pelo neurônio,  $\hat{f}$ 
8       fim
9     fim
10    Calcular  $erro_{parcial} = y - \hat{f}$ 
11    para cada camada da rede, a partir da camada de saída faça
12      para cada neurônio  $n_{jl}$  da camada atual faça
13        Ajustar pesos do neurônio utilizando Equação 7.3
14      fim
15    fim
16    Calcular  $erro_{total} = erro_{total} + erro_{parcial}$ 
17  fim
18 até  $erro_{total} < \xi$ ;
```

Começamos com **pesos aleatórios**
Modelo “nasce sem saber nada”

Backpropagation – Treinamento

Algoritmo 7.2 Algoritmo de treinamento *back-propagation*

Entrada: Um conjunto de n objetos de treinamento

Saída: Rede MLP com valores dos pesos ajustados

```
1 Inicializar pesos da rede com valores aleatórios
2 Inicializar  $erro_{total} = 0$ 
3 repita
4   para cada objeto  $x_i$  do conjunto de treinamento faça
5     para cada camada da rede, a partir da primeira camada intermediária faça
6       para cada neurônio  $n_{jl}$  da camada atual faça
7         Calcular valor da saída produzida pelo neurônio,  $\hat{f}$ 
8       fim
9     fim
10    Calcular  $erro_{parcial} = y - \hat{f}$ 
11    para cada camada da rede, a partir da camada de saída faça
12      para cada neurônio  $n_{jl}$  da camada atual faça
13        Ajustar pesos do neurônio utilizando Equação 7.3
14      fim
15    fim
16    Calcular  $erro_{total} = erro_{total} + erro_{parcial}$ 
17  fim
18 até  $erro_{total} < \xi$ ;
```

Backpropagation – Treinamento

Algoritmo 7.2 Algoritmo de treinamento *back-propagation*

Entrada: Um conjunto de n objetos de treinamento

Saída: Rede MLP com valores dos pesos ajustados

```
1 Inicializar pesos da rede com valores aleatórios
2 Inicializar  $erro_{total} = 0$ 
3 repita
4   para cada objeto  $x_i$  do conjunto de treinamento faça
5     para cada camada da rede, a partir da primeira camada intermediária faça
6       para cada neurônio  $n_{jl}$  da camada atual faça
7         Calcular valor da saída produzida pelo neurônio,  $\hat{f}$ 
8       fim
9     fim
10    Calcular  $erro_{parcial} = y - \hat{f}$ 
11    para cada camada da rede, a partir da camada de saída faça
12      para cada neurônio  $n_{jl}$  da camada atual faça
13        Ajustar pesos do neurônio utilizando Equação 7.3
14      fim
15    fim
16    Calcular  $erro_{total} = erro_{total} + erro_{parcial}$ 
17  fim
18 até  $erro_{total} < \xi$ ;
```

Repetir as duas fases até o meu erro total ficar **suficientemente pequeno**

Backpropagation – Treinamento

Algoritmo 7.2 Algoritmo de treinamento *back-propagation*

Entrada: Um conjunto de n objetos de treinamento

Saída: Rede MLP com valores dos pesos ajustados

```
1 Inicializar pesos da rede com valores aleatórios
2 Inicializar  $erro_{total} = 0$ 
3 repita
4   para cada objeto  $x_i$  do conjunto de treinamento faça
5     para cada camada da rede, a partir da primeira camada intermediária faça
6       para cada neurônio  $n_{jl}$  da camada atual faça
7         Calcular valor da saída produzida pelo neurônio,  $\hat{f}$ 
8       fim
9     fim
10    Calcular  $erro_{parcial} = y - \hat{f}$ 
11    para cada camada da rede, a partir da camada de saída faça
12      para cada neurônio  $n_{jl}$  da camada atual faça
13        Ajustar pesos do neurônio utilizando Equação 7.3
14      fim
15    fim
16    Calcular  $erro_{total} = erro_{total} + erro_{parcial}$ 
17  fim
18 até  $erro_{total} < \xi$ ;
```

A rede aprende um exemplo por vez

Backpropagation – Treinamento

Algoritmo 7.2 Algoritmo de treinamento *back-propagation*

Entrada: Um conjunto de n objetos de treinamento

Saída: Rede MLP com valores dos pesos ajustados

1 Inicializar pesos da rede com valores aleatórios

2 Inicializar $erro_{total} = 0$

3 **repita**

4 **para cada** objeto $\mathbf{x}_i$ do conjunto de treinamento **faça**

5 **para cada** camada da rede, a partir da primeira camada intermediária **faça**

6 **para cada** neurônio n_{jl} da camada atual **faça**

7 Calcular valor da saída produzida pelo neurônio, $\hat{f}$

8 **fim**

9 **fim**

10 Calcular $erro_{parcial} = y - \hat{f}$

11 **para cada** camada da rede, a partir da camada de saída **faça**

12 **para cada** neurônio n_{jl} da camada atual **faça**

13 Ajustar pesos do neurônio utilizando Equação 7.3

14 **fim**

15 **fim**

16 Calcular $erro_{total} = erro_{total} + erro_{parcial}$

17 **fim**

18 **até** $erro_{total} < \xi$;

Passo para frente
(forward)

Backpropagation – Treinamento

Algoritmo 7.2 Algoritmo de treinamento *back-propagation*

Entrada: Um conjunto de n objetos de treinamento

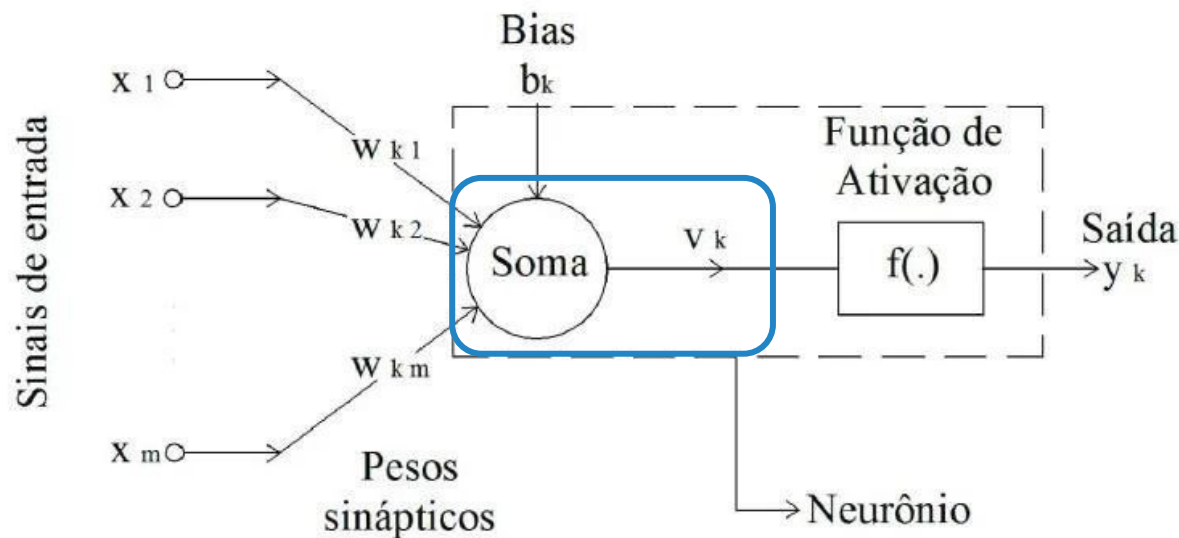
Saída: Rede MLP com valores dos pesos ajustados

```
1 Inicializar pesos da rede com valores aleatórios
2 Inicializar  $erro_{total} = 0$ 
3 repita
4   para cada objeto  $x_i$  do conjunto de treinamento faça
5     para cada camada da rede, a partir da primeira camada intermediária faça
6       para cada neurônio  $n_{jl}$  da camada atual faça
7         Calcular valor da saída produzida pelo neurônio,  $\hat{f}$ 
8       fim
9     fim
10    Calcular  $erro_{parcial} = y - \hat{f}$ 
11    para cada camada da rede, a partir da camada de saída faça
12      para cada neurônio  $n_{jl}$  da camada atual faça
13        Ajustar pesos do neurônio utilizando Equação 7.3
14      fim
15    fim
16    Calcular  $erro_{total} = erro_{total} + erro_{parcial}$ 
17  fim
18 até  $erro_{total} < \xi$ ;
```

Cada neurônio calcula sua saída
Usa pesos + função de ativação
Resultado final: previsão da rede ($\hat{f}$)

Backpropagation – Treinamento

Relembrando o cálculo da saída de um neurônio



$$v_k = \sum_{j=1}^m w_{kj} x_j - b_k$$

Backpropagation – Treinamento

Algoritmo 7.2 Algoritmo de treinamento *back-propagation*

Entrada: Um conjunto de n objetos de treinamento

Saída: Rede MLP com valores dos pesos ajustados

1 Inicializar pesos da rede com valores aleatórios

2 Inicializar $erro_{total} = 0$

3 **repita**

4 **para cada** objeto $\mathbf{x}_i$ do conjunto de treinamento **faça**

5 **para cada** camada da rede, a partir da primeira **faça**

6 **para cada** neurônio n_{jl} da camada atual **faça**

7 Calcular valor da saída produzida pelo neurônio

8 **fim**

9 **fim**

10 Calcular $erro_{parcial} = y - \hat{f}$

11 **para cada** camada da rede, a partir da camada de saída **faça**

12 **para cada** neurônio n_{jl} da camada atual **faça**

13 Ajustar pesos do neurônio utilizando Equação 7.3

14 **fim**

15 **fim**

16 Calcular $erro_{total} = erro_{total} + erro_{parcial}$

17 **fim**

18 **até** $erro_{total} < \xi$;

No fim, calcula o erro para o objeto em questão do conjunto de treinamento

Backpropagation – Treinamento

Algoritmo 7.2 Algoritmo de treinamento *back-propagation*

Entrada: Um conjunto de n objetos de treinamento

Saída: Rede MLP com valores dos pesos ajustados

```
1 Inicializar pesos da rede com valores aleatórios
2 Inicializar  $erro_{total} = 0$ 
3 repita
4   para cada objeto  $x_i$  do conjunto de treinamento faça
5     para cada camada da rede, a partir da primeira camada intermediária faça
6       para cada neurônio  $n_{jl}$  da camada atual faça
7         Calcular valor da saída produzida pelo neurônio,  $\hat{f}$ 
8       fim
9     fim
10    Calcular  $erro_{parcial} = y - \hat{f}$ 
11    para cada camada da rede, a partir da camada de saída faça
12      para cada neurônio  $n_{jl}$  da camada atual faça
13        Ajustar pesos do neurônio utilizando Equação 7.3
14      fim
15    fim
16    Calcular  $erro_{total} = erro_{total} + erro_{parcial}$ 
17  fim
18 até  $erro_{total} < \xi$ ;
```

Passo para trás
(backward)

Backpropagation – Treinamento

Algoritmo 7.2 Algoritmo de treinamento *back-propagation*

Entrada: Um conjunto de n objetos de treinamento

Saída: Rede MLP com valores dos pesos ajustados

```
1 Inicializar pesos da rede com valores aleatórios
2 Inicializar  $erro_{total} = 0$ 
3 repita
4   para cada objeto  $x_i$  do conjunto de treinamento faça
5     para cada camada da rede, a partir da primeira camada intermediária faça
6       para cada neurônio  $n_{jl}$  da camada atual faça
7         Calcular valor da saída produzida pelo neurônio,  $\hat{f}$ 
8       fim
9     fim
10    Calcular  $erro_{parcial} = y - \hat{f}$ 
11    para cada camada da rede, a partir da camada de saída faça
12      para cada neurônio  $n_{jl}$  da camada atual faça
13        Ajustar pesos do neurônio utilizando Equação 7.3
14      fim
15    fim
16    Calcular  $erro_{total} = erro_{total} + erro_{parcial}$ 
17  fim
18 até  $erro_{total} < \xi$ ;
```

Para cada neurônio,
ajusta os pesos

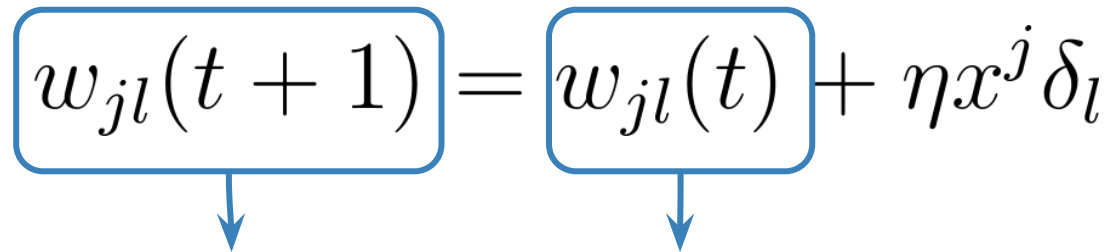
Perceptron – Treinamento

Ajuste de pesos:

$$w_{jl}(t + 1) = w_{jl}(t) + \eta x^j \delta_l$$

Perceptron – Treinamento

Ajuste de pesos:

$$\boxed{w_{jl}(t + 1)} = \boxed{w_{jl}(t)} + \eta x^j \delta_l$$


Peso atualizado

Entre um neurônio l
e o j -ésimo neurônio
da camada anterior

Peso atual

Entre um neurônio l
e o j -ésimo neurônio
da camada anterior

Perceptron – Treinamento

Ajuste de pesos:

$$w_{jl}(t + 1) = w_{jl}(t) + \eta x^j \delta_l$$

Peso atualizado
Entre um neurônio l
e o j -ésimo neurônio
da camada anterior

Peso atual
Entre um neurônio l
e o j -ésimo neurônio
da camada anterior

Learning rate
(taxa de aprendizado)

Perceptron – Treinamento

Ajuste de pesos:

$$w_{jl}(t + 1) = w_{jl}(t) + \eta x^j \delta_l$$

Peso atualizado
Entre um neurônio l
e o j -ésimo neurônio
da camada anterior

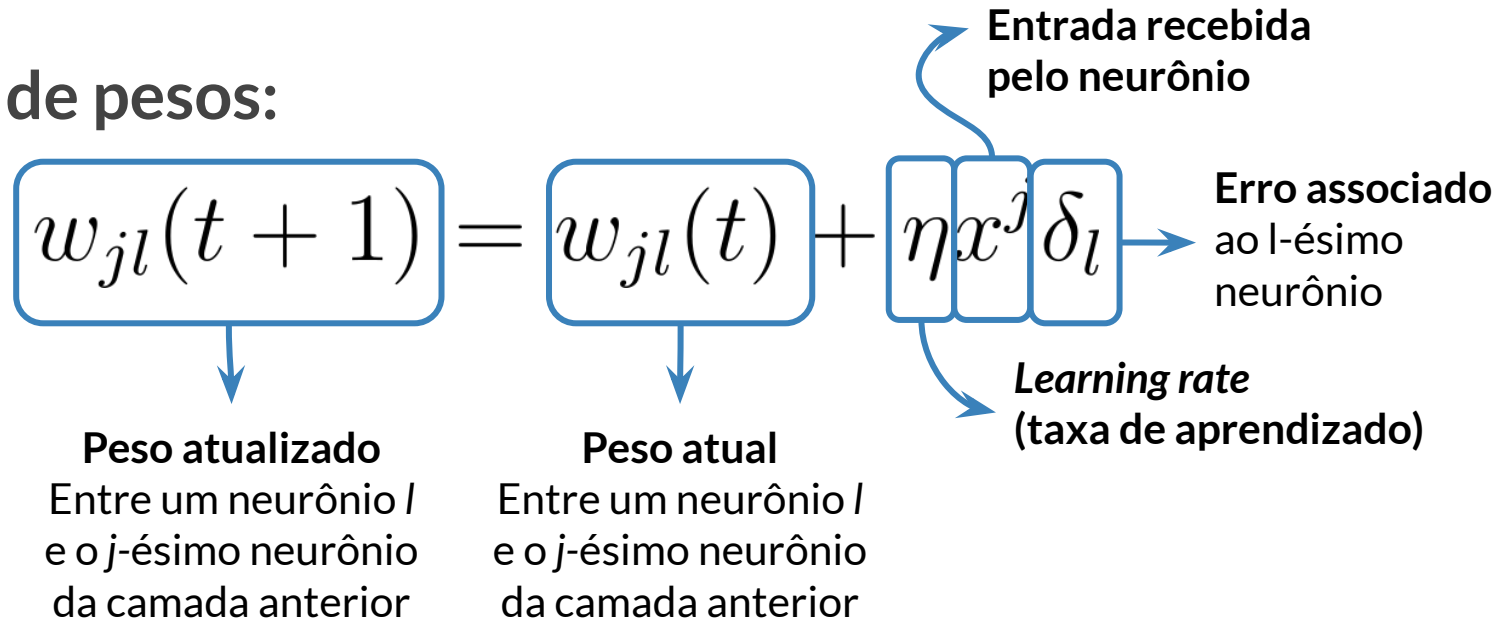
Peso atual
Entre um neurônio l
e o j -ésimo neurônio
da camada anterior

**Entrada recebida
pelo neurônio**

**Learning rate
(taxa de aprendizado)**

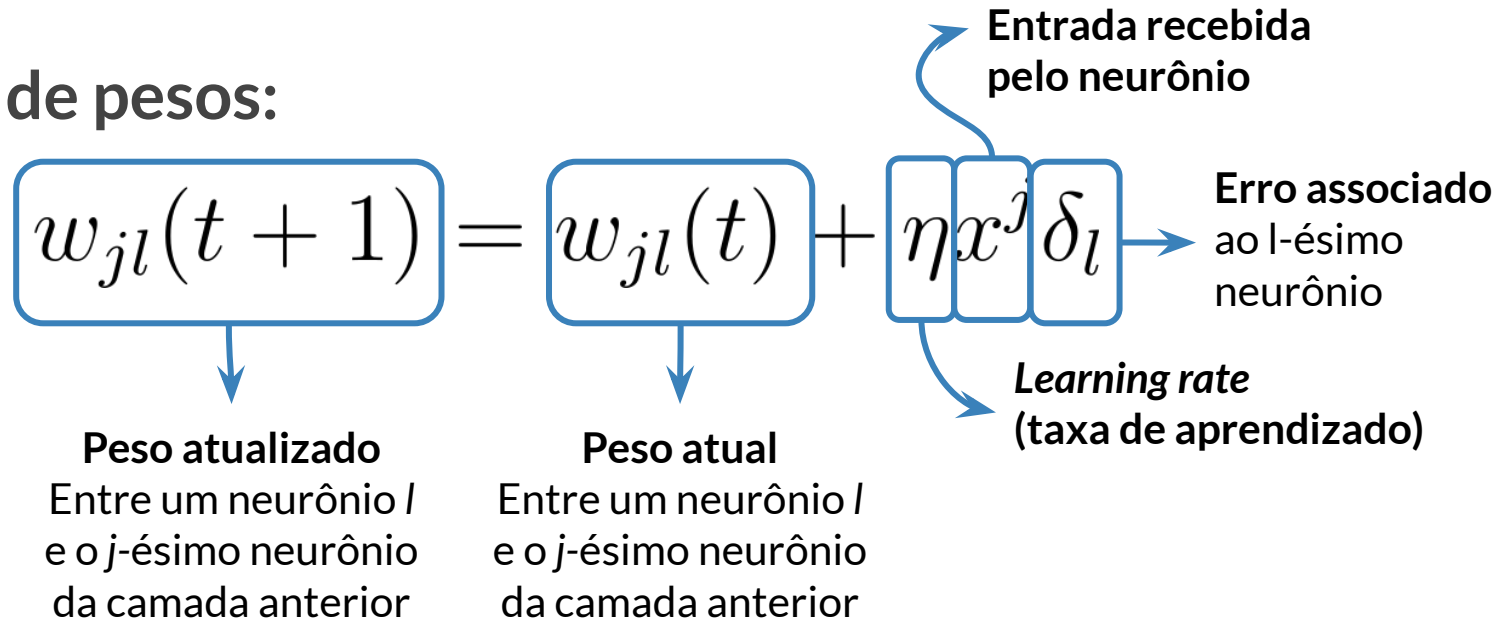
Perceptron – Treinamento

Ajuste de pesos:



Perceptron – Treinamento

Ajuste de pesos:

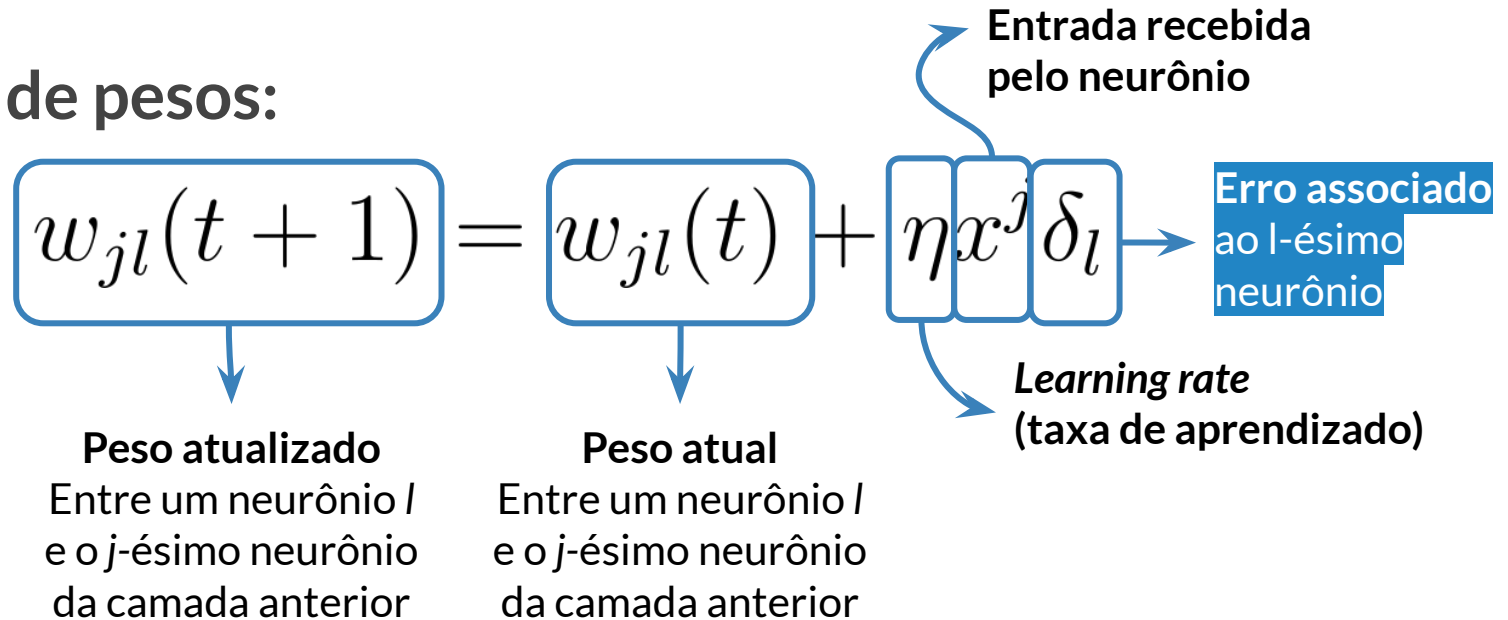


Intuição:

Novo peso = peso antigo + (taxa de aprendizado * entrada * erro)

Perceptron – Treinamento

Ajuste de pesos:



Backpropagation – Treinamento

Sobre o cálculo do erro:

- ▷ Erros são conhecidos apenas para os neurônios da camada de saída
 - É preciso estimar o erro dos neurônios das camadas intermediárias!
- ▷ **Intuição** → somar os erros dos neurônios da camada seguinte ponderados pelo valor do peso de cada conexão

Backpropagation – Treinamento

Sobre o cálculo do erro:

- ▷ Depende então da camada

$$\delta_l = \begin{cases} f'_l e_l, & \text{se } n_l \in C_{saida} \\ f'_l \sum w_{lk} \delta_k, & \text{se } n_l \in C_{intermediaria} \end{cases}$$

Backpropagation – Treinamento

Sobre o cálculo do erro:

- ▷ Depende então da camada

Derivada parcial da função de ativação

$$\delta_l = \begin{cases} f'_l e_l, & \text{se } n_l \in C_{saida} \\ f'_l \sum w_{lk} \delta_k, & \text{se } n_l \in C_{intermediaria} \end{cases}$$

Backpropagation – Treinamento

Sobre o cálculo do erro:

- ▷ Depende então da camada

Se for camada de saída, e_l é o erro quadrático médio (MSE)

$$\delta_l = \begin{cases} f'_l e_l, & \text{se } n_l \in C_{saída} \\ f'_l \sum w_{lk} \delta_k, & \text{se } n_l \in C_{intermediária} \end{cases}$$

Backpropagation – Treinamento

Sobre o cálculo do erro:

- ▷ Depende então da camada

$$e_l = \frac{1}{2} \sum_{q=1}^k (y_1 - \hat{f}_q)^2$$

Se for camada de saída, e_l é o erro quadrático médio (MSE)

$$\delta_l = \begin{cases} f'_l e_l, & \text{se } n_l \in C_{saída} \\ f'_l \sum w_{lk} \delta_k, & \text{se } n_l \in C_{intermediária} \end{cases}$$

Backpropagation – Treinamento

Sobre o cálculo do erro:

- ▷ Depende então da camada

$$\delta_l = \begin{cases} f'_l e_l, & \text{se } n_l \in C_{saida} \\ f'_l \sum w_{lk} \delta_k, & \text{se } n_l \in C_{intermediaria} \end{cases}$$



Se for camada intermediária,
calculamos a soma dos erros dos
neurônios da camada seguinte
ponderados pelos pesos

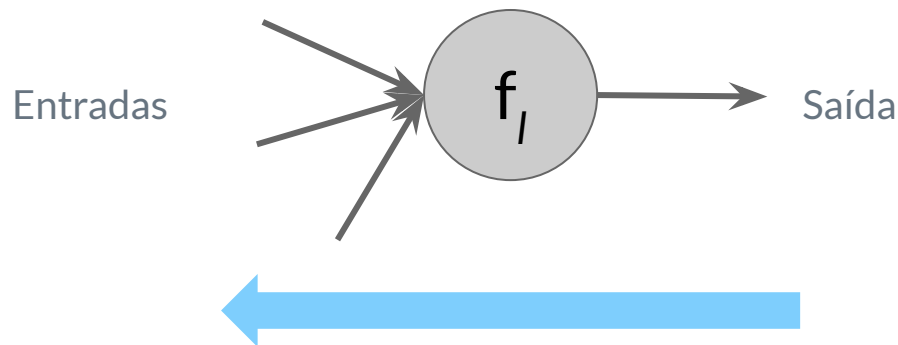
Backpropagation – Treinamento

Sobre o cálculo do erro:

- ▷ Depende então da camada

$$\delta_l = \begin{cases} f'_l e_l, & \text{se } n_l \in C_{saida} \\ f'_l \sum w_{lk} \delta_k, & \text{se } n_l \in C_{intermediaria} \end{cases}$$

Derivada parcial da função de ativação



Backpropagation – Treinamento

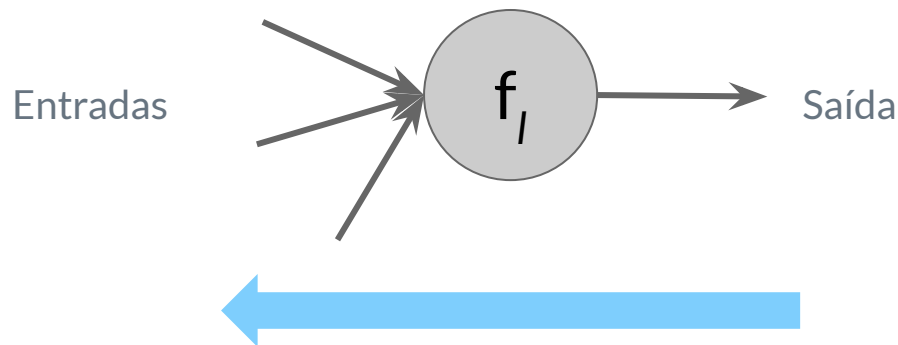
Sobre o cálculo do erro:

- ▷ Depende então da camada

Derivada parcial define ajuste usando **gradiente descendente**

Derivada parcial da função de ativação

$$\delta_l = \begin{cases} f'_l e_l, & \text{se } n_l \in C_{saida} \\ f'_l \sum w_{lk} \delta_k, & \text{se } n_l \in C_{intermediaria} \end{cases}$$



3.

Gradiente Descendente

A bússola para saber como diminuir o erro

Gradiente Descendente – Intuição

O backpropagation é um método de **otimização baseada em gradiente**

- Os pesos são ajustados calculando como o erro varia quando cada peso muda um pouco

Gradiente descendente → depende das derivadas para saber a **direção e a intensidade** da mudança no erro

- Sem ele, não seria possível decidir se aumentamos ou diminuimos o peso

Gradiente Descendente – Intuição

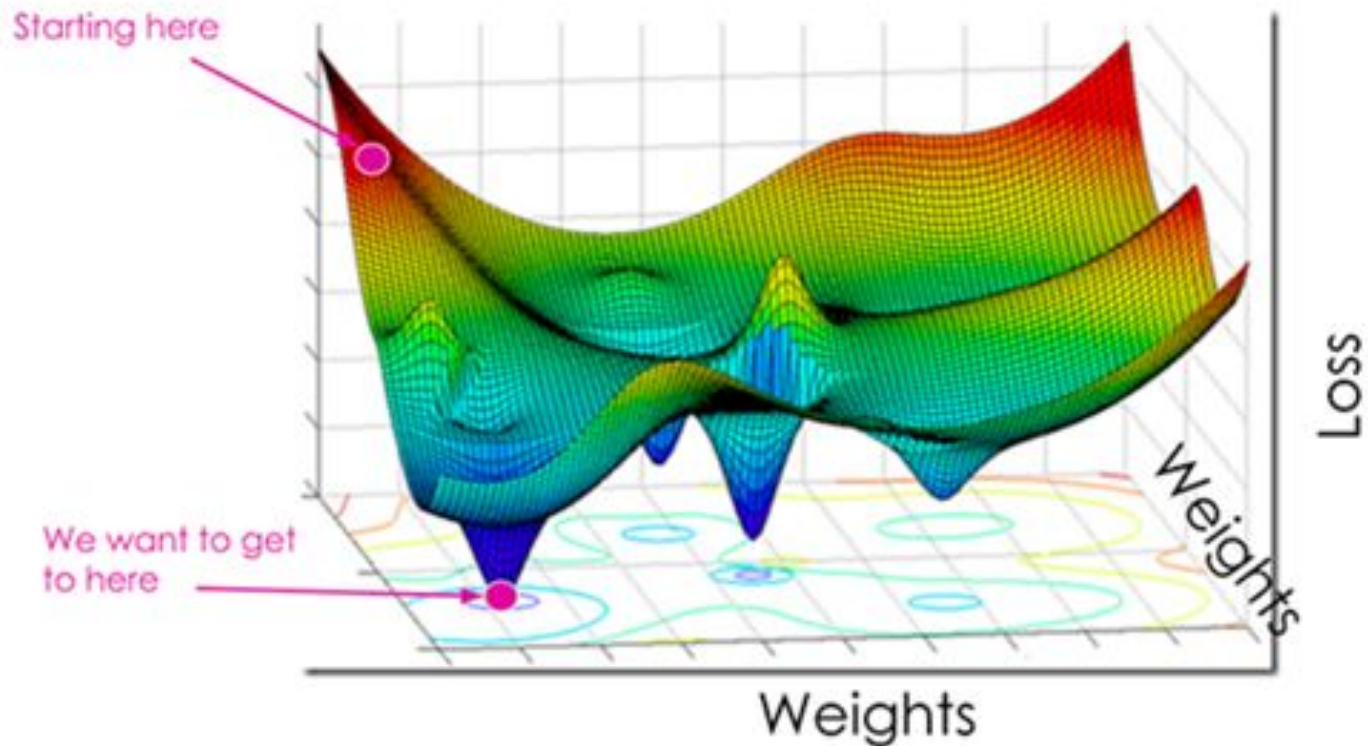
Por que precisamos de derivadas?

A derivada nos diz: “Se eu mudar este peso um pouquinho, o erro aumenta ou diminui?”

- **Derivada positiva** → peso está aumentando o erro → devemos diminuí-lo (o peso)
- **Derivada negativa** → peso está diminuindo o erro → devemos aumentá-lo (o peso)
- **Derivada zero** → chegamos a um ponto estável (mínimo)

A derivada é calculada para cada peso da rede

Gradiente Descendente – Intuição



Extraído de:

<https://www.analyticsvidhya.com/blog/2023/01/gradient-descent-vs-backpropagation-whats-the-difference/>

Gradiente Descendente – Intuição

Por que precisamos de derivadas?

Conclusão → O sinal negativo da derivada é o que permite ao gradiente “subir” ou “descer” na direção certa

- É isso que faz o gradiente descendente funcionar!

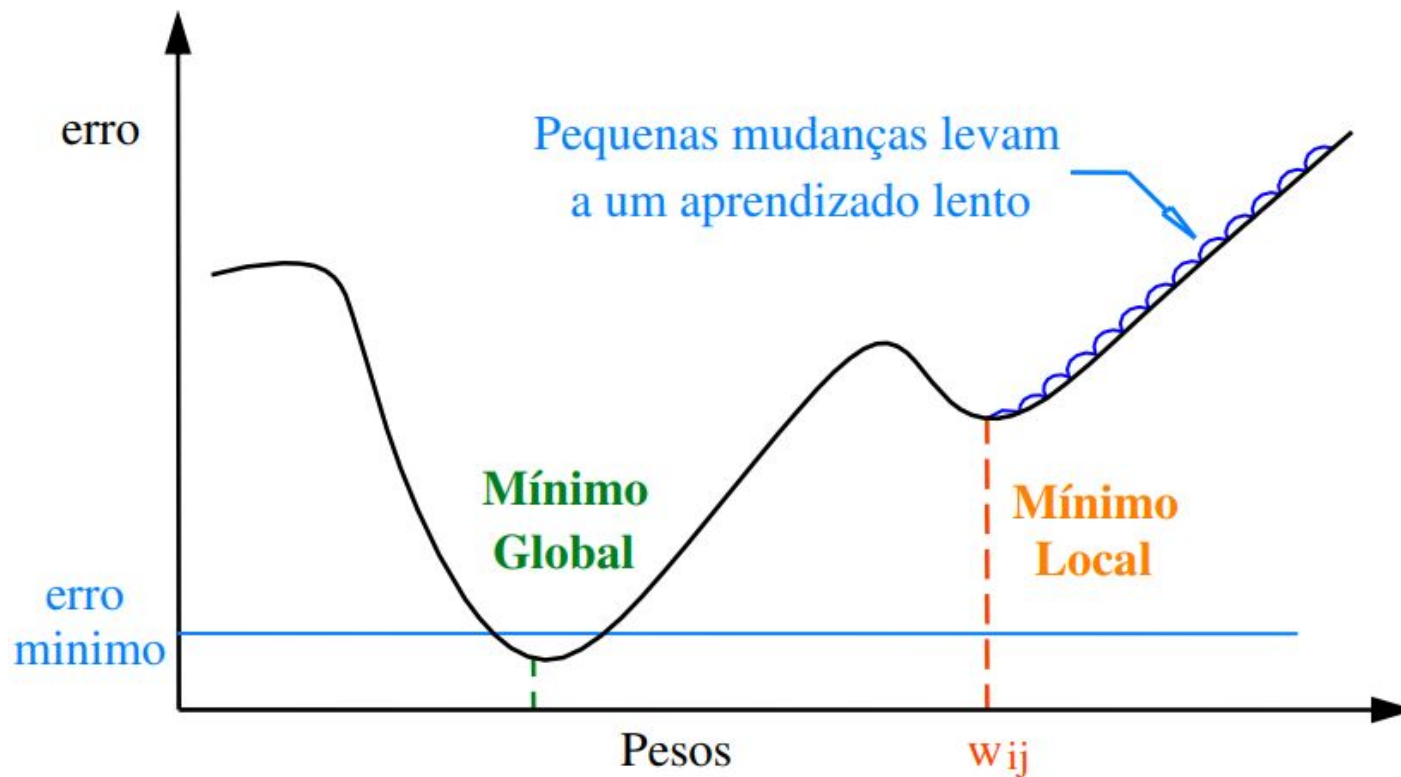
4.

Taxa de Aprendizado

Como afeta na otimização dos pesos?

Taxa de Aprendizado

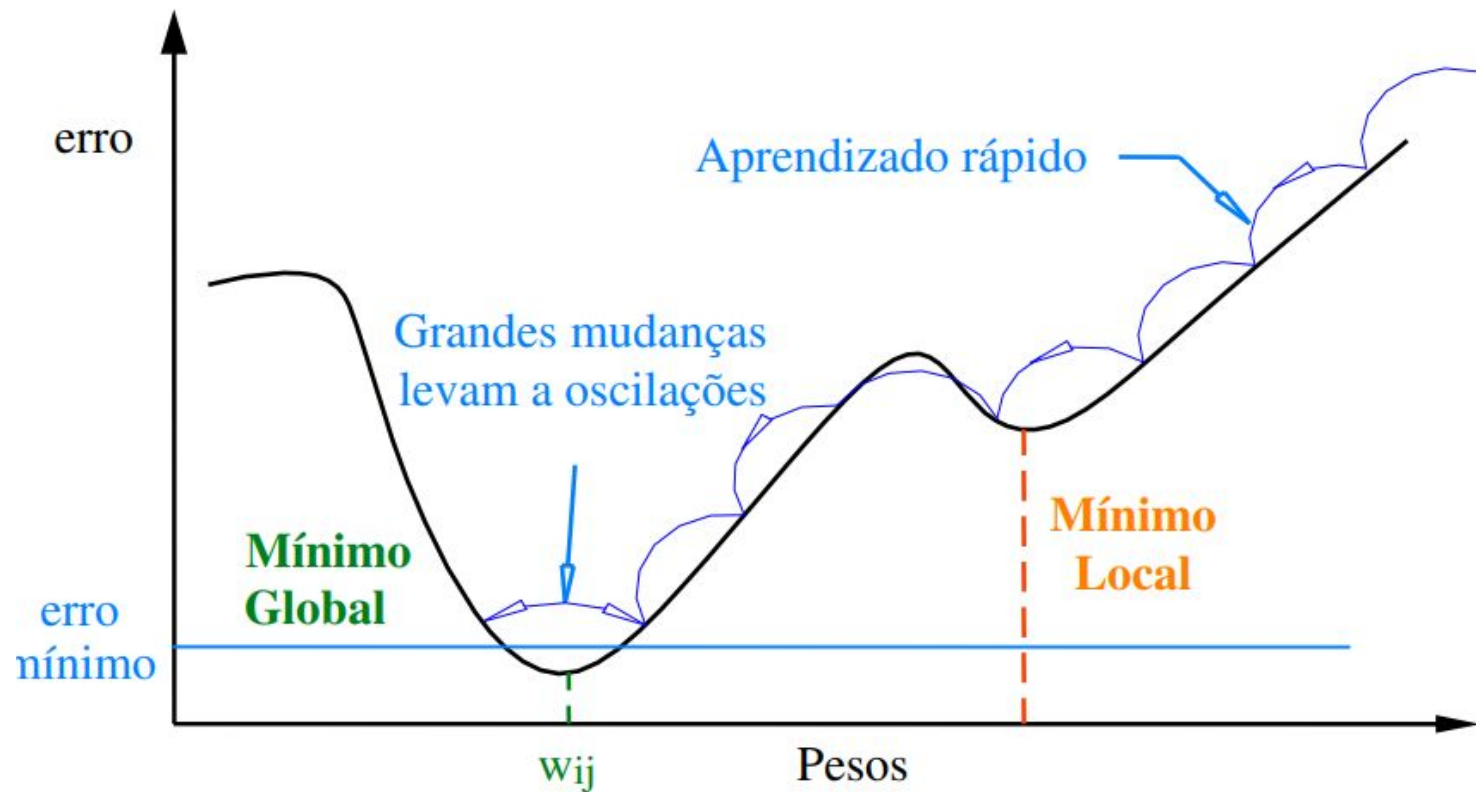
Learning rate (η) $\rightarrow$ Tamanho do passo



Extraído de: https://www.maxwell.vrac.puc-rio.br/32823/32823_4.PDF

Taxa de Aprendizizado

Learning rate (η) $\rightarrow$ Tamanho do passo



Extraído de: https://www.maxwell.vrac.puc-rio.br/32823/32823_4.PDF

5.

Funções de Ativação

A mágica das redes neurais acontece aqui

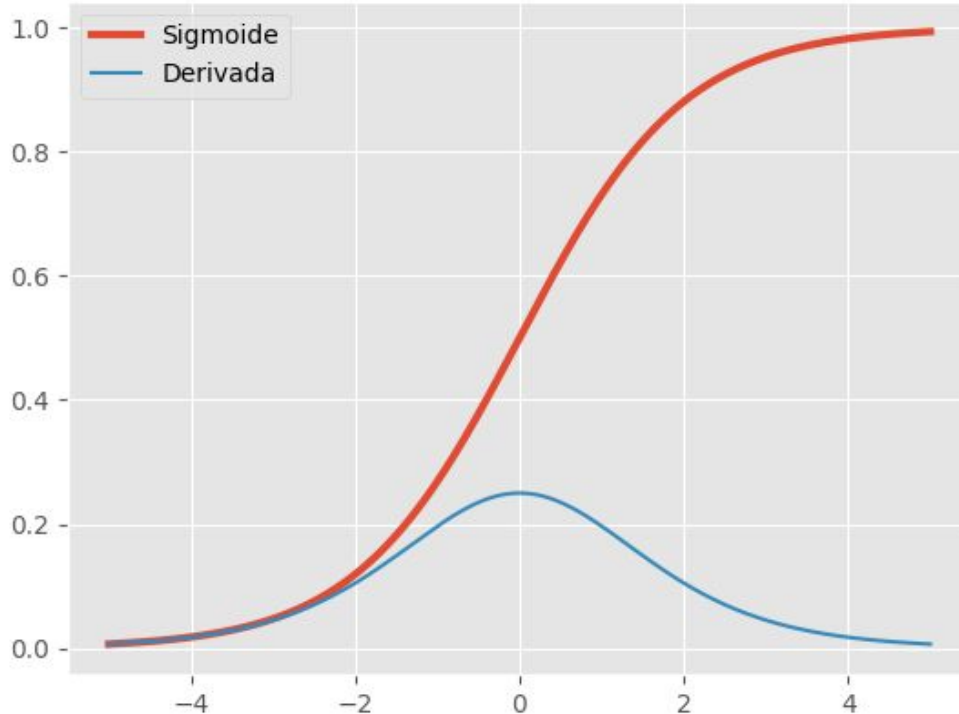
Funções de Ativação

- ▷ É o que dá à rede a capacidade de aprender coisas não-lineares
- ▷ Sem ela, empilhar camadas seria inútil
 - Equivalente a uma rede de uma camada só
- ▷ Como utilizamos gradiente descendente, precisamos de **funções diferenciáveis**

Funções de Ativação

Sigmoide

$$\sigma(x) = \frac{1}{1 + e^x} \quad \sigma'(x) = \sigma(x)(1 - \sigma(x))$$



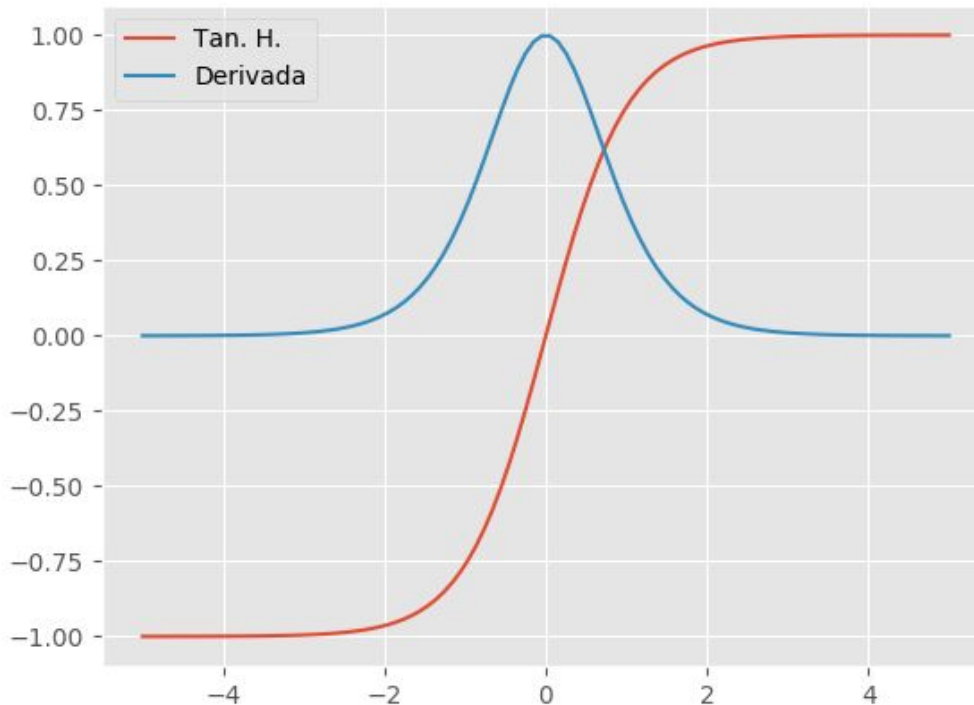
Saída entre 0 e 1 (útil na camada de saída como classificação binária)

Problema → gradiente some em redes profundas (*vanishing gradient*)

Funções de Ativação

Tangente Hiperbólica

$$\tanh(x) = 2\sigma(2x) - 1 \quad \tanh'(x) = 1 - \tanh^2(x)$$



Saída entre -1 e 1

Melhor que sigmoide em
muitos casos

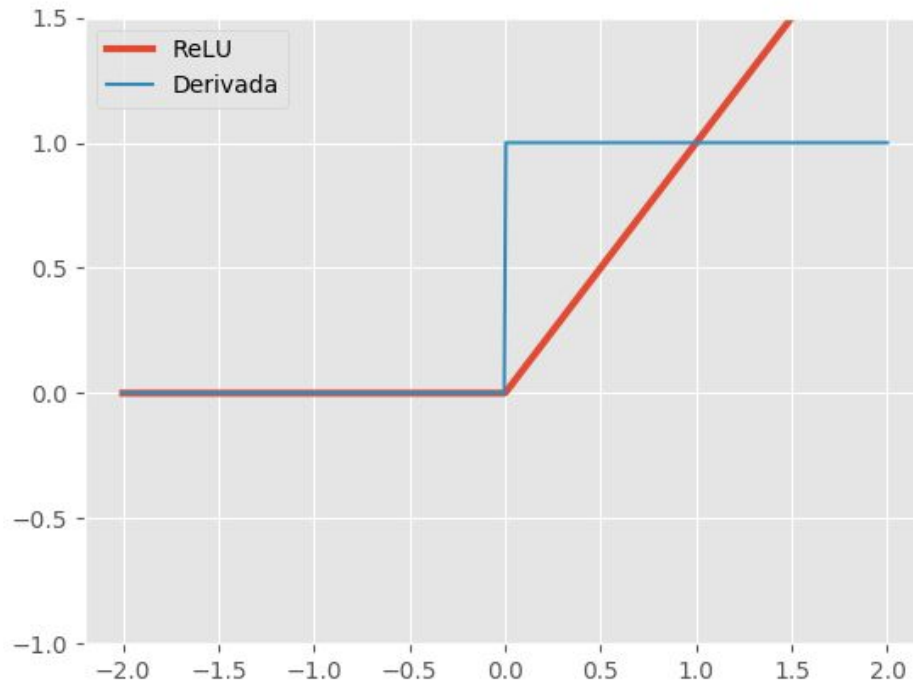
Funções de Ativação

ReLU

(Ativação Linear Retificada)

$$\text{ReLU}(x) = \max\{0, x\}$$

$$\text{ReLU}'(x) = \begin{cases} 1, & \text{se } x \geq 0 \\ 0, & \text{c.c.} \end{cases}$$



Derivada simples (0 ou 1)

Muito usada hoje em dia

Funções de Ativação

- ▷ Existem outras funções utilizadas
 - SoftPlus, SoftMax, etc.

Para saber mais:

<https://matheusfacure.github.io/2017/07/12/activ-func/>

6.

Outras Considerações

Informações úteis

Variações do algoritmo

Normalmente, há duas formas de atualização dos pesos:

- ▷ **Stochastic Gradient Descent (Padrão)**
 - Atualiza pesos após cada exemplo
 - Rápido, mas instável
- ▷ **Batch Gradient Descent**
 - Atualiza pesos depois de ver todos os exemplos
 - Estável, mas lento

Variações do algoritmo

Epoch (época) → cada vez que há a apresentação completa de todos os dados de treino

Heurísticas para n° de neurônios

Valor médio → o número de neurônios da camada escondida (intermediária) é a média do número de entradas e saídas da rede:

$$q = \frac{p + M}{2}$$

Heurísticas para n° de neurônios

Raíz quadrada → o número de neurônios da camada escondida (intermediária) é a raiz quadrada do produto do número de entradas e saídas da rede:

$$q = \sqrt{p \cdot M}$$

Heurísticas para n° de neurônios

Kolmogorov → o número de neurônios da camada escondida (intermediária) é o dobro do número de entradas mais 1:

$$q = 2p + 1$$

Outras sugestões:

- ▷ Estar entre o número de entrada e saída
- ▷ 2/3 o tamanho da entrada + tamanho da saída
- ▷ Menor que o dobro do tamanho da entrada

Referências

FACELI, Katti et al. **Inteligência artificial: uma abordagem de aprendizado de máquina**. Rio de Janeiro, RJ: LTC, 2011. (**Capítulo 7**)

Material da Profa. Cristiane Nobre

Outros links:

<https://matheusfacure.github.io/2017/07/12/activ-func/>

https://www.maxwell.vrac.puc-rio.br/32823/32823_4.PDF

<https://www.analyticsvidhya.com/blog/2023/01/gradient-descent-vs-backpropagation-whats-the-difference/>